

MoEGambit: Contract-Based Hybrid Recovery for Mixture-of-Experts Training

Anonymous Author 1

*Affiliation omitted
for double-anonymous review*

Anonymous Author 2

*Affiliation omitted
for double-anonymous review*

Abstract—Large-scale Mixture-of-Experts (MoE) training runs for months on thousands of GPUs, and rank failures are common. Existing mechanisms typically recover by restarting from a globally consistent checkpoint. In MoE layouts where expert data parallelism (EDP) equals one, this restart treats replicated non-expert state and rank-unique expert state as one monolithic object. That restart path discards GPU work already completed after the checkpoint: to return from checkpoint step c to failure step t , the job reloads old state and replays $t - c$ iterations, even though current state may survive on healthy peers. Dense peer recovery exploits this opportunity when the failed rank’s full state has a live replica, but MoE training at $\text{EDP} = 1$ breaks that assumption: only non-expert state is peer-recoverable, while expert state is rank-unique. We present MoEGambit, a recovery-side framework that subsumes dense peer recovery when all state is replicated and extends it to heterogeneous MoE state. When a runtime contract permits hybrid repair, MoEGambit pulls current non-expert state from a healthy dense data-parallel (dense-DP) peer and loads only the failed rank’s expert shard from checkpoint, eliminating replay and reducing checkpoint I/O to the state that lacks a live peer. The contract combines a safe-point repair controller, an expert-weighted staleness density $\Phi'(t)$ that quantifies stale-expert exposure for gating partial repair, and a guarded reintegration state machine with structured logs. A weights-first/optimizer-later two-phase protocol further overlaps optimizer restoration with resumed computation. In our experiments on Qwen3-30B-A3B and DeepSeek-V2-Lite, MoEGambit reduced raw recovery latency by 20.6%–55.0% and yielded a $36.9\times$ replay-inclusive speedup for a 100-iteration replay gap. In tested contract-allowed scenarios, paired evaluation found no statistically detectable degradation in validation loss, perplexity, or downstream zero-shot accuracy.

Index Terms—Large language models, mixture-of-experts, fault tolerance, distributed training, failure recovery, checkpointing, runtime monitoring

I. INTRODUCTION

Large language model (LLM) training is a long-running, multi-thousand-GPU software process, so failures are an expected operating condition rather than a rare exception. Llama 3 reports 419 unexpected interruptions during a 54-day, 16,384-GPU pre-training period, roughly one every three hours [1]; MegaScale documents production failures and stragglers in a 10K-GPU deployment [2]; and the OPT-175B logbook records prolonged human intervention during training [3]. The standard recovery mechanism, *checkpoint restart*, reloads the latest globally consistent snapshot and replays all lost iterations, regardless of which state was actually invalidated. This all-or-nothing rollback wastes the GPU time

spent after the checkpoint; at modern failure rates, even modest per-failure outages accumulate into substantial aggregate GPU time over a training run.

This replay cost increasingly affects sparse Mixture-of-Experts (MoE) training runs for models such as Mixtral [4], DeepSeek-V2/V3 [5], [6], Qwen3-MoE [7], Ling (Bailing) [8], Ring-flash-linear-2.0 [9], and Kimi K2 [10]. FlashRecovery [11] shows how dense training can avoid some of this cost: if the failed rank’s full state is replicated, a replacement can reconstruct it from a healthy data-parallel peer. MoE training changes the recovery assumption. At $\text{EDP} = 1$, only the *non-expert* state (attention, embedding, router) is replicated in this way, while expert parameters and optimizer moments are rank-unique after expert sharding. Restart ignores this split: it rolls back the whole job, reloads all checkpointed state, and replays every lost iteration.

Prior fault-tolerant training systems improve pieces of this recovery path: they reduce checkpoint cost [12]–[17], adapt topology around failures [18]–[20], or recover dense replicas from peers [11]. However, they do not address MoE’s recovery-side asymmetry at $\text{EDP} = 1$. Checkpoint/replica systems preserve checkpoint-consistent restart semantics; topology-adaptation systems rely on rank interchangeability; dense peer recovery assumes that the failed rank’s full state has a live peer. With $\text{EDP} = 1$, that assumption fails for experts: non-expert state can be peer-recovered, but expert state has no live peer and must come from checkpoint. When $\text{EDP} > 1$, expert replicas exist and dense-style peer recovery can also repair expert state; our distinct target is the $\text{EDP} = 1$ operating point that prior peer-recovery methods cannot cover. This boundary matters because recent MoE models and production systems use many experts and wide expert parallelism to scale model capacity while reducing communication cost. Under Megatron Core’s MoE parallelism mapping, increasing EP or expert tensor parallelism (ETP) under a fixed GPU budget drives $\text{EDP} = W/(\text{PP} \times \text{EP} \times \text{ETP})$ toward one [6], [7], [21], [22]. MoE-specific systems have begun to optimize checkpoint/log-based recovery: MoC-System [23] reduces save-side cost through Partial Experts Checkpointing, and MoEvent [24] uses sparse checkpointing with log-assisted localized recovery. **This leaves a recovery-side gap:** no existing system combines peer recovery for replicated MoE state with checkpoint repair for rank-unique experts while bounding the quality risk introduced by partial repair.

Key observation. While peer recovery is well established for dense training, MoE’s heterogeneous state makes recovery only *partially* peer-recoverable at $\text{EDP} = 1$. Non-expert parameters (attention, embedding, router) are replicated across the dense-DP dimension and can be pulled from a healthy dense-DP peer, just as in dense peer recovery. Expert parameters and optimizer state, however, are EP-sharded and have no live peer at $\text{EDP} = 1$. When a rank fails, the replacement can therefore pull current-step non-expert state from a healthy dense-DP peer and load only the rank-local expert shard from the latest checkpoint—**eliminating replay while reducing disk I/O** to the rank-unique expert volume. At $\text{EDP} > 1$, MoEGambit uses the same state-class boundary but can restore expert state from a surviving expert peer, matching the easier setting assumed by prior peer-recovery systems.

Hybrid recovery raises a software-engineering question beyond I/O cost. It resumes with current non-expert state but checkpoint-stale expert state, so the restored experts lag the rest of the model by $\Delta = t - c$ iterations; repeated recoveries can accumulate this staleness across the expert population. Without a runtime-checkable specification of when the hybrid path is admissible, operators face an unaudited tradeoff between speed and model quality. Following the self-adaptive-systems view of monitored, auditable runtime adaptation [25]–[27], MoE recovery should be specified against the post-recovery training trajectory rather than only against process liveness.

We present **MoEGambit**, a recovery-side framework for sparse MoE training. MoEGambit allows hybrid recovery only after checking and logging expert-staleness exposure. It introduces an **explicit recovery contract** with three artifacts: a safe-point repair controller that prevents partial optimizer commits (R1), an expert-weighted staleness density $\Phi'(t)$ with an empirically calibrated quality threshold (R2), and a guarded reintegration state machine with structured logs (R3). MoEGambit then contributes **two recovery mechanisms**: a MoE-aware hybrid repair path that pulls current non-expert parameters from a healthy dense-DP peer and loads only the failed rank’s expert shard from checkpoint, and a weights-first/optimizer-later two-phase recovery protocol that overlaps optimizer restoration with resumed computation.

We evaluated MoEGambit on Qwen3-30B-A3B [7] (128 experts, 64 H20 GPUs) and DeepSeek-V2-Lite [5] (64 experts) under software engineering (SE) experimentation standards: every recovered run was paired with a NoFault baseline sharing the same seed, data order, checkpoint, and injected failure step [28], [29], so residual quality differences could be attributed specifically to recovery rather than to seed noise. MoEGambit reduced raw recovery latency by 20.6%–55.0% and avoided replay under the hybrid path; for the 100-iteration replay gap in RQ1, this yielded a $36.9\times$ replay-inclusive speedup. In tested contract-allowed scenarios, paired tests showed no statistically detectable degradation in validation loss or perplexity; in the 50-run single-failure heatmap, 44/50 runs fell within the NoFault $\pm 2\sigma_{\text{base}}$ band, and downstream zero-shot accuracy remained indistinguishable from Restart

while staying within the NoFault noise scale.

This paper makes three contributions:

- **Runtime recovery contract.** Three software-engineering artifacts—safe-point invariant (R1), expert-weighted staleness density $\Phi'(t)$ with an empirically calibrated quality threshold (R2), and a guarded reintegration state machine with structured audit logs (R3)—that separate *when* hybrid repair is allowed from *how* state is reconstituted (§III).
- **Hybrid repair and two-phase recovery.** A recovery-side mechanism that pulls dense-DP-replicated non-expert state from a healthy peer at step t and loads only EP-sharded expert state from a per-rank checkpoint shard at step c , plus a weights-first/optimizer-later protocol that overlaps optimizer restoration with resumed training (§III-E–§III-F).
- **Paired fault-injection evaluation.** A reproducible benchmark on Qwen3-30B-A3B and DeepSeek-V2-Lite with paired NoFault baselines following SE experimentation guidelines [28], [29], demonstrating 20.6%–55.0% latency reduction, $36.9\times$ replay-inclusive speedup for a 100-iteration gap, and no statistically detectable degradation in tested scenarios (§IV).

Table I positions MoEGambit against the closest fault-tolerant training families.

II. BACKGROUND AND MOTIVATION

We first define the MoE state classes that make partial recovery possible, then show why checkpoint restart leaves this structure unused.

A. Concepts: MoE Training as a Heterogeneous-State Distributed Program

A distributed MoE training job [22], [30]–[34] is a long-running, stateful, multi-process program executing collective communication on tens to thousands of GPUs. Modern MoE frameworks may use different mappings for dense/attention layers and MoE layers: dense layers use tensor, context, pipeline, and data parallelism, while MoE layers use expert parallelism (EP), expert tensor parallelism (ETP), and expert data parallelism (EDP). Unlike dense Transformers, MoE training maintains a *heterogeneous state* with three recovery-relevant classes:

- **Replicated state.** Non-expert layers (attention, embedding, layer normalization) and router weights are replicated across every dense-DP rank that owns the same PP/dense-TP coordinate. At any iteration t , at least one healthy peer holds the current-step value.
- **Sharded expert state.** Expert weights and their AdamW first/second moments are partitioned across EP and ETP groups. In a 128-expert, $\text{EP}=8$, $\text{ETP}=1$ layout without expert replication, each rank exclusively owns a fixed subset of $128/8 = 16$ experts; no peer holds a current in-memory replica.
- **Runtime metadata.** The expert directory, the dispatch topology used by all-to-all routing [35], [36], and the

TABLE I
POSITIONING AGAINST REPRESENTATIVE SYSTEMS. “PARTIAL” IS SCOPED TO A FAMILY OR STATE SUBSET; “LIMITED” LACKS A QUALITY GUARD.

Representative system/family	Live/log repair	MoE-aware	Replay-free	Runtime quality guard	Structured audit trail
Checkpoint/replica systems [12]–[17]	No	No	No	No	Limited
Topology adaptation [18]–[20]	Yes	No	Yes	No	Limited
FlashRecovery (dense peer recovery) [11]	Yes	No	Yes	No	Limited
MoE-specific checkpoint/log systems [23], [24]	Partial	Yes	Partial	No	Limited
MoEGambit	Yes	Yes	Yes	Yes	Yes

process-group view are derived from the rank layout and become stale the moment any rank is replaced.

Expert Data Parallelism (EDP). Following Megatron Core’s current MoE terminology [22], we distinguish dense tensor parallelism (TP) from expert tensor parallelism (ETP). Dense layers are organized by TP, CP, PP, and dense-DP, whereas MoE layers are organized by PP, EP, ETP, and EDP. Because our experiments did not use context parallelism, the expert layout satisfies

$$W = PP \times EP \times ETP \times EDP, \quad EDP = \frac{W}{PP \times EP \times ETP}.$$

This differs from older EP-as-a-subdimension-of-DP descriptions, where TP was sometimes included in the EDP denominator. Under the current mapping, dense TP affects attention-layer sharding but does not by itself determine expert replication; ETP is the MoE-layer tensor-parallel degree. When $EDP = 1$, every expert shard exists on exactly one live rank; when $EDP > 1$, a failed expert shard may have a surviving peer copy, analogous to the peer copy used for non-expert parameters. $EDP = 1$ is the most constrained recovery operating point because every expert shard is rank-unique. In our evaluated Qwen3-30B-A3B layout, $W = 64$, $PP = 8$, $EP = 8$, and $ETP = 1$, so $EDP = 64/(8 \cdot 8 \cdot 1) = 1$ even though the model has 128 experts; each EP rank owns 16 experts within its pipeline stage. This operating point is resource-driven in large-expert MoE systems [6], [7], [21]: large expert counts already distribute memory across ranks, while extra expert replication multiplies expert and optimizer memory without increasing activated compute because each token activates only $K \ll E$ experts. It is also the setting in which MoE recovery differs from dense recovery: the expert portion of a failed rank has no live peer and must be restored from checkpoint. Fig. 1 illustrates this distinction.

The software-engineering consequence is that **the same failure invalidates different state classes with different recovery semantics**; standard checkpoint restart does not exploit this boundary.

B. Existing Flow: Checkpoint Restart and the Hidden Risk of Partial Recovery

Motivating example. In our 64-GPU Qwen3-30B-A3B job, a failure detected at step $t = 200$ with the latest checkpoint at $c = 100$ causes standard Megatron-LM to reload a 56 GB distributed checkpoint and replay 100 iterations, for an outage

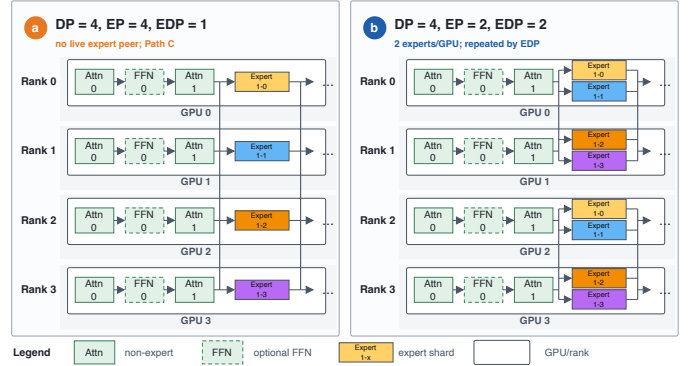


Fig. 1. Expert distribution with 8 GPUs (dense TP=1, ETP=1, PP=2). With $EDP > 1$, experts have live peers; with $EDP = 1$, each expert shard is unique and must be recovered from checkpoint. Non-expert parameters (green) are dense-DP-replicated in both layouts.

of ~ 1067 s. The failed rank, however, owns only 16 of 128 experts; non-expert layers and the other 112 experts remain live on healthy ranks. A MoE-aware recovery path can therefore pull 3.2 GB of non-expert state from a peer, load only the failed rank’s expert shard, and resume from step t in 29 s with no replay. This opportunity also introduces a quality risk: the recovered experts are stale by $\Delta = t - c$ iterations.

Production LLM training commonly uses *checkpoint restart* [1], [2], [37]: on a failure, the job reloads the most recent globally consistent snapshot at step c and replays the $t - c$ lost iterations. Classical rollback-recovery and checkpoint-interval analyses [38]–[40] show that the optimal checkpoint interval scales as $\tau^* \approx \sqrt{2CM}$, where C is the per-checkpoint cost and M is the mean time between failures; with M collapsing to tens of minutes on 10K-GPU clusters [1], [2], repeated failures can consume substantial aggregate GPU-hours in checkpoint I/O and replay.

Restart treats the entire job as a single all-or-nothing state object. Table II shows why this is wasteful: a single failed rank invalidates only its EP-sharded expert state, while the dense-DP-replicated non-expert state of every other rank at the same PP/dense-TP coordinate remains current at step t . A natural alternative is *hybrid recovery*: synchronize non-expert state from a healthy dense-DP peer at the current step, and load only the rank-local experts from the latest checkpoint shard at step c . Hybrid recovery, however, introduces a side

TABLE II
PROVENANCE OF MOE TRAINING STATE AT ITERATION t IN THE EDP= 1
RECOVERY CASE.

State	Layout	Source
Non-expert (attn/embed/router)	Dense-DP	Peer (step t)
Expert weights	EP/ETP	Checkpoint shard ($c < t$)
Expert optimizer	EP/ETP	Checkpoint shard ($c < t$)
Runtime metadata	Rank-derived	Recomputed

effect that restart does not: the recovered rank’s expert state is stale by $\Delta = t - c$ iterations relative to the rest of the model.

This staleness is not a binary defect—restored experts still produce valid forward/backward signals—but it can perturb the load-balancing dynamics driven by the MoE auxiliary loss [31], [41]–[43]. The risk has two dimensions: a *per-event* gap Δ for the experts owned by the failed rank, and a *window-level* debt accumulated by repeated hybrid recoveries across different expert shards. A recovery system that allows partial repair therefore needs a runtime-checkable bound on accumulated staleness, not only a fast mechanism.

Problem statement. MoEGambit receives a fail-stop event $\langle r, t, c \rangle$ from the cluster watchdog, where r is the failed logical rank, c is the latest checkpoint, and t is the first step that should execute after safe-point handling. If a failure is detected mid-iteration, R1 discards that in-flight iteration and advances t to the next valid step; thus $\Delta = t - c$ counts the iterations that checkpoint restart would replay. MoEGambit must choose either RESTART, which restores all state from checkpoint and replays $t - c$ iterations, or HYBRID, which restores current non-expert state from a dense-DP peer and expert state from the failed rank’s checkpoint shard. In both cases, the first post-recovery `optimizer.step()` must be a valid AdamW update: no gradient may be applied to uninitialized or partially restored parameters.

Requirements. We formulate the recovery path as a contract with three software-engineering requirements. **R1** defines a repair point and gates optimizer commits so a mid-step failure cannot corrupt model state. **R2** specifies hybrid eligibility through a quantitative, $O(1)$ guard that captures per-event and accumulated staleness. **R3** records every decision, guard input, latency segment, and state-machine transition for post-hoc auditability. MoEGambit assumes fail-stop rank failures, at least one available checkpoint, a healthy dense-DP peer for the failed rank’s non-expert state, and a replacement/hot-spare GPU; if the peer condition fails, the policy chooses RESTART.

III. APPROACH

A. Overview

MoEGambit is a runtime recovery layer that intercepts rank-failure events and reconstructs the replacement rank’s state from the most current available source for each state class, without rolling back the rest of the job. Fig. 2 summarizes the runtime architecture. The design separates *what is guaranteed* (the recovery contract, R1–R3) from *how state is repaired* (the

hybrid restoration mechanism and two-phase protocol), so the contract can be audited independently of the implementation.

The framework executes five components on each failure event (Fig. 2):

- 1) *Failure detection.* An external watchdog emits a failure event $\langle r, t, c \rangle$ identifying the failed rank, the current training step, and the latest checkpoint step.
- 2) *Safe-point repair controller* (§III-B, R1). Installs an optimizer commit guard so no rank applies gradients computed during recovery.
- 3) *Staleness-density guarded policy* (§III-C, R2). Evaluates the $O(1)$ guard $\Phi'(t)$ and returns a deterministic decision $\pi(t) \in \{\text{HYBRID}, \text{RESTART}\}$. If the projected staleness density exceeds the empirical quality threshold, MoEGambit falls back to checkpoint restart.
- 4) *MoE-aware hybrid restoration and two-phase protocol* (§III-E–§III-F). Pulls non-expert state from a healthy dense-DP peer at step t (path P), loads rank-local experts from the checkpoint shard at step c (path C), and overlaps optimizer-state restoration with resumed computation.
- 5) *Reintegration state machine and structured logs* (§III-G, R3). Advances the replacement rank to HEALTHY and emits a structured record linking policy inputs, latencies, and state-machine transitions.

B. Safe-Point Repair Controller (R1)

R1 defines a single program point at which repair may run, with one invariant: no optimizer commit may use gradients computed under RECOVERING. On a failure event $\langle r, t, c \rangle$, the controller atomically marks r and its experts as RECOVERING, installs a pre-AdamW commit guard on every rank, and marks the current iteration as DISCARD. Any rank reaching `optimizer.step()` under DISCARD skips the update; in-flight gradients are dropped. Training then resumes through hybrid repair (§III-E) or restart, and the controller logs the safe-point status.

C. Staleness-Density Guarded Policy (R2)

R2 defines an $O(1)$ guard $\Phi'(t)$ and a deterministic decision $\pi(t) \in \{\text{HYBRID}, \text{RESTART}\}$. The decision uses the failure event $\langle r, t, c \rangle$, the window staleness debt $S(t) = \sum_{h \in \mathcal{H}_{W_{\text{exp}}}(t)} |E_h| \Delta_h$, the predicate $\text{PEERAVAIL}(r, t)$, and thresholds $(\Delta_{\min}, \Delta_{\max}, \Phi_{\max})$ calibrated in §IV. Here W_{exp} is the exposure-window length, $\mathcal{H}_{W_{\text{exp}}}(t)$ is the set of previous hybrid events in $[t - W_{\text{exp}}, t)$, E_h is the expert set restored by event h , Δ_h is its checkpoint gap, and N_{expert} is the routed-expert count in the exposure domain; thus $S(t)$ has units of expert-iterations. If the current event were recovered through the hybrid path, the projected density would be

$$\Phi'(t) = \frac{S(t) + |E_{\text{new}}| \Delta}{N_{\text{expert}} \cdot W_{\text{exp}}}, \quad (1)$$

where $N_{\text{expert}} \cdot W_{\text{exp}}$ is the debt of exposing all experts for one full window; $\Phi'(t) = 0.1$ therefore means a projected exposure equal to 10% of that full-window budget. The value is an

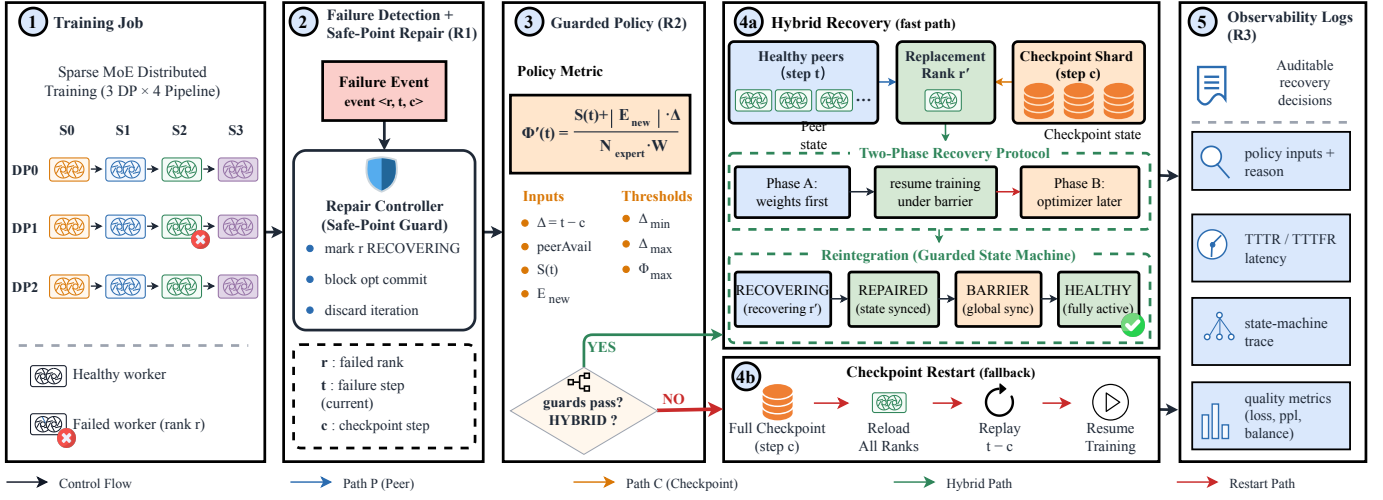


Fig. 2. MoEGambit runtime architecture. On a rank failure, R1 establishes a repair point, R2 chooses HYBRID or RESTART, the hybrid path restores replicated and expert state from different sources, and R3 logs the guarded reintegration trace.

exposure density, not a probability, and can exceed 1 under repeated exposure within the window. The running aggregate $S(t)$ is maintained as a single scalar: add $|E_h|\Delta_h$ when a hybrid event h enters the window, and subtract the same product when that event leaves the window (older than W_{exp} iterations). This yields $O(1)$ per-iteration bookkeeping. When multiple ranks fail simultaneously, each failed rank contributes its own $|E_h|\Delta_h$ term to $S(t)$ before the policy is evaluated for the next rank.

Intuition. The guard is a monitorable exposure budget, not a claim of global convergence. A hybrid event exposes $|E_h|$ experts to weights that are Δ_h iterations behind the rest of the model; repeated events expose more experts for longer. Charging each event by $|E_h|\Delta_h$ captures both factors, and normalizing by $N_{\text{expert}} \cdot W_{\text{exp}}$ makes $\Phi'(t)$ comparable across model sizes and window lengths. The threshold $\Phi_{\text{max}} = 10^{-1}$ (§IV) is calibrated against the observed failure-free noise floor.

The decision is

$$\pi(t) = \begin{cases} \text{HYBRID,} & \text{if } \text{PEERAVAIL}(r, t) \wedge \\ & \Delta_{\min} \leq \Delta \leq \Delta_{\max} \wedge \\ & \Phi'(t) \leq \Phi_{\max}, \\ \text{RESTART,} & \text{otherwise.} \end{cases} \quad (2)$$

Algorithm 1 encodes the same logic and records the first failing guard on every fallback. The three thresholds play distinct roles. $\Delta_{\min}$ is a cost-benefit lower bound that avoids hybrid repair when the replay gap is too small to offset its coordination cost; $\Delta_{\max}$ is a per-event staleness cap on any single recovered expert shard; and $\Phi_{\max}$ is a window-level cap on accumulated expert exposure. Each threshold is tied to a measurable source: $\Delta_{\min}$ to the $T_{\text{load}} : T_{\text{hybrid}} : T_{\text{iter}}$ ratio, $\Delta_{\max}$ to the checkpoint interval bounded by Young/Daly [39], [40], $\Phi_{\max}$ to the guard-disabled quality sweep, and PEERAVAIL to the live process group. The log entry contains $\langle \pi(t), \text{reason} \rangle$

and $(\Delta, |E_{\text{new}}|, S(t), \Phi'(t))$ (§III-G); the compact reason labels identify peer unavailability, the two gap guards, the exposure guard, and the allowed hybrid path.

D. Theoretical Rationale for the Guard

The recovery contract is the monitorable admission condition under which MoEGambit may choose the fast path. For every recovery event, hybrid recovery implies all three guards:

$$\begin{aligned} \pi(t) = \text{HYBRID} \Rightarrow & \text{PEERAVAIL}(r, t) \\ & \wedge \Delta_{\min} \leq \Delta \leq \Delta_{\max} \\ & \wedge \Phi'(t) \leq \Phi_{\max}, \quad \forall t. \end{aligned} \quad (3)$$

R3 logs evidence for each conjunct, so the contract is checkable online and auditable offline. The threshold $\Phi_{\max}$ is empirical, but the form of $\Phi'(t)$ follows from a first-order exposure bound.

Algorithm 1 Guarded recovery decision $\pi(t)$.

Input: event $\langle r, t, c \rangle$; debt $S(t)$; constants $N_{\text{expert}}, W_{\text{exp}}$

Input: guards $(\Delta_{\min}, \Delta_{\max}, \Phi_{\max})$; PEERAVAIL

Output: recovery decision and first-failing reason

```

1:  $\Delta \leftarrow t - c$ ;  $e \leftarrow |E_{\text{new}}(r)|$ 
2:  $\rho \leftarrow (S(t) + e\Delta) / (N_{\text{expert}} W_{\text{exp}})$ 
3: if  $\neg \text{PEERAVAIL}(r, t)$  then
4:   return (RESTART, NoPEER)
5: else if  $\Delta < \Delta_{\min}$  then
6:   return (RESTART, SMALLGAP)
7: else if  $\Delta > \Delta_{\max}$  then
8:   return (RESTART, LARGE GAP)
9: else if  $\rho > \Phi_{\max}$  then
10:  return (RESTART, HIGHDEBT)
11: else
12:  return (HYBRID, ADMIT)
13: end if

```

Proposition 1 (expert-exposure bound). Let $\theta_e(s)$ denote expert e ’s state after step s . Assume that, within a recovery window and with non-expert state fixed, the monitored MoE loss component ℓ_{aux} is coordinate-wise L_{aux} -Lipschitz in each expert state, and a normal step changes any expert by at most G in the same norm. If event h restores expert set E_h from checkpoint step c_h and resumes at t_h , then its stale-expert perturbation relative to a current-state restore satisfies $\delta\ell_h \leq L_{\text{aux}} \sum_{e \in E_h} \|\theta_e(t_h) - \theta_e(c_h)\| \leq L_{\text{aux}} G |E_h| (t_h - c_h)$. Consequently, after adding the current candidate event to the existing window debt, $\sum_{h \in \mathcal{H}_{W_{\text{exp}}}(t) \cup \{\text{new}\}} \delta\ell_h \leq L_{\text{aux}} G (S(t) + |E_{\text{new}}| \Delta) = L_{\text{aux}} G N_{\text{expert}} W_{\text{exp}} \Phi'(t)$. Thus $\Phi'(t)$ is a normalized exposure proxy whose induced first-order perturbation bound scales linearly with $\Phi'(t)$; it is not a standalone convergence theorem.

E. MoE-Aware Hybrid State Restoration

When $\pi(t) = \text{HYBRID}$, the replacement rank reconstructs its state by reading each state class from its most current available source (Table II), so I/O scales with the rank-unique expert volume rather than the full distributed checkpoint. Two paths run concurrently. *Path P (peer pull, replicated state)* synchronizes current-step non-expert parameters (and dense-DP-replicated optimizer state) from a healthy peer at the same PP/dense-TP coordinate via a single P2P broadcast over NCCL/Gloo; the transfer completes in tens of milliseconds. *Path C (shard read, sharded expert state)* loads rank- r ’s experts from the per-rank shard at step c with no collectives. The recovered rank is intentionally mixed: non-expert state reflects step t , while experts and their optimizer state are stale by Δ —the quantity bounded by $\Phi'(t)$ in (1) (Fig. 2, step 4a).

F. Two-Phase Recovery Protocol

The two-phase protocol reduces time-to-resume by overlapping path C’s optimizer-state read with resumed forward/backward, while forbidding commits on uninitialized expert slots. *Phase A (weights first)* restores non-expert state and expert weights, then rejoins under an *update barrier*: forward/backward contributes to the global loss, but gradients for affected experts are buffered. *Phase B (optimizer later)* restores non-expert and expert moments in parallel; when it completes, the barrier releases and buffered gradients are applied via one AdamW step. The barrier reuses the pre-step() hook of §III-B, so R1 holds throughout. The log records T_{TTR} and T_{TTFR} ; their gap is the latency saved over single-phase restart. Because the protocol controls *when* optimizer state attaches, not *where* bytes come from, §IV-D’s 2×2 factorial tests it independently of hybrid restore.

G. Reintegration State Machine and Structured Logs (R3)

R3 requires that every recovery decision and its downstream model-quality outcome be traceable. We implement R3 through two mechanisms: a guarded state machine that enforces the reintegration protocol, and a structured log that records the inputs and outputs of each recovery event.

Guarded state machine. Each replacement rank advances through a four-state linear protocol:

$$\text{RECOVERING} \xrightarrow{g_1} \text{REPAIRED} \xrightarrow{g_2} \text{BARRIER} \xrightarrow{g_3} \text{HEALTHY}, \quad (4)$$

where each transition is gated by a guard predicate:

- g_1 : state restoration complete—all state classes (non-expert via path P, experts via path C) have been loaded and verified against their source checksums.
- g_2 : dispatch topology re-derived—the expert directory and all-to-all routing tables have been recomputed to reflect the replacement rank’s position in the EP layout.
- g_3 : optimizer state attached and update barrier released—Phase B of the two-phase protocol (§III-F) has completed, and the buffered gradients have been applied.

Because every transition requires its guard to hold, the state machine rejects premature reintegration—e.g., routing tokens to an expert whose optimizer state has not yet been restored.

Structured recovery log. On each recovery event, the state machine emits one record with policy inputs (Δ , $|E_{\text{new}}|$, $S(t)$, $\Phi'(t)$), the decision and reason string, per-segment latencies, and transition timestamps. This record links every recovery path back to the guards in (2) and supports post-hoc correlation with training-quality metrics under the paired-run methodology of §IV [28], [29]. The overhead of $\Phi'(t)$ accounting and log emission is $O(1)$ per iteration and empirically negligible (§IV-G).

H. Implementation

MoEGambit is implemented as a Python extension to Megatron-LM (commit `core_r0.9.0`) covering safe-point control, policy evaluation, hybrid restore, two-phase optimizer recovery, and reintegration. It hooks optimizer-step, checkpoint-load, and collective paths while preserving standard checkpoint restart; without `--enable-moegambit`, training follows the standard path. The anonymized supplementary material contains the full system code, structured logs, and plotting/reproduction scripts; we will make the artifacts public after the double-anonymous review period.

IV. EVALUATION

This section evaluates whether MoEGambit reduces recovery cost without degrading the post-recovery training trajectory. We ask six research questions:

- **RQ1:** How much recovery cost does MoEGambit reduce, and do hybrid restore and two-phase recovery compose additively?
- **RQ2:** Does hybrid recovery permitted by the contract avoid detectable quality degradation relative to failure-free training?
- **RQ3:** Does $\Phi'(t)$ separate low-exposure recoveries from risky repeated and burst failures?
- **RQ4:** What overhead do the monitoring hooks add during failure-free training?
- **RQ5:** Does the recovery advantage hold across scale and parallelism layouts?

- **RQ6:** Does the mechanism generalize to a different MoE model configuration?

A. Experimental Setup

Cluster and parallelism. Unless otherwise stated, experiments ran on 64 H20-3e GPUs (8 nodes $\times$ 8 GPUs) with dense TP= 1, PP= 8, EP= 8, ETP= 1, and EDP= 1 under the Megatron Core definition. The scalability and parallelism-sensitivity experiments used the overrides described in RQ5.

Model and tokenizer. We used *Qwen3-30B-A3B* [7] (30B total / 3B activated, 48 layers, 128 experts, top-8 routing) with the Qwen2Tokenizer (151,936-token byte-level BPE). Default training hyperparameters are in Table III.

Data. We used a 4B-token subset of *FineWeb* [44], processed into Megatron indexed format. We used a held-out validation split for loss/perplexity reported every 1,000 iterations. All recovery experiments used *paired runs*: baseline and MoEGambit shared the same seed, data order, checkpoint, and injected failure step.

Baseline timing and noise scale. Steady-state $T_{\text{iter}} \approx 10.31$ s; full-checkpoint load $T_{\text{load}} \approx 35.95$ s; hybrid recovery $T_{\text{hybrid}} \approx 28.91$ s; `save-interval= 200` (Young/Daly optimal [39], [40]). Ten NoFault runs yielded $\mu_{\text{base}} = 4.8543$, $\sigma_{\text{base}} = 0.024$ on iter-600 eval loss; we used σ_{base} to scale deviations and reported both $\pm 1\sigma_{\text{base}}$ and $\pm 2\sigma_{\text{base}}$ bands. This noise scale captures residual variation from collective-communication timing, node/GPU heterogeneity, shared-storage jitter, and nondeterministic kernel scheduling that remains even with fixed seeds and data order.

Metrics. Raw recovery latency excludes replay and ends when the replacement rank reaches HEALTHY; time-to-resume (T_{TTR}) ends at the first valid post-recovery forward pass; time-to-full-recovery (T_{TFR}) ends when optimizer state is attached and the update barrier is released. End-to-end recovery additionally includes replay for Restart and MoC-System; MoEGambit’s hybrid path resumes at step t and has zero replay cost.

Statistical treatment. Latency experiments reported the mean over 500 injected recovery events because recovery cost is dominated by deterministic I/O and collective-transfer segments; quality experiments used paired comparisons to control for seed and data-order noise. We used NoFault σ_{base} bands as practical effect-size references and paired nonparametric

tests to assess degradation; we did not require every individual recovered run to lie inside the $\pm 1\sigma_{\text{base}}$ band.

B. Compared Systems

We compared MoEGambit with three baselines: **NoFault** (no injected failures, reference trajectory); **Restart** (standard Megatron-LM checkpoint restart with full reload and replay); and **MoC-System** (a published MoE-specific baseline with enough detail for timing and quality reproduction [23]). We discuss MoEvment [24] in Related Work because it relies on sparse checkpointing plus activation/gradient logging, whereas MoEGambit uses peer-current non-expert repair without logging intermediate activations or gradients.

MoC-System. MoC-System is a save-side MoE fault-tolerance system that reduces checkpointing cost with Partial Experts Checkpointing (PEC): each checkpoint stores only a selected subset of experts and rotates that subset across saves. Because no public reference implementation is available, we separated accuracy and timing. For accuracy, we reproduced the best PEC configuration reported in the MoC-System paper ($K_{\text{pec}} = 16$, $N = 128$, $\text{PLT} \approx 3.75\%$) by redirecting each non-fresh expert to its corresponding historical checkpoint shard, with MoEGambit’s hybrid repair and two-phase protocol disabled. For timing, we used the best recovery numbers reported by, or derivable from, the paper; since MoC-System is save-side, recovery still resumes from checkpoint state and replays lost iterations to reach step t .

C. Policy Parameters

MoEGambit uses three thresholds: Δ_{min} , Δ_{max} , Φ_{max} (Table IV).

Δ_{min} and Δ_{max} were non-binding under our timings ($T_{\text{load}} > 1.2 T_{\text{hybrid}}$, so hybrid has lower latency for every $\Delta \geq 1$); we retain both as defensive guards for clusters with much faster checkpoint loading.

$\Phi_{\text{max}} = 10^{-1}$ **calibration.** We calibrated the threshold with a guard-disabled 3×4 multi-rank burst sweep (Table V) to identify the quality-degradation boundary that the default policy should avoid. We set $W_{\text{exp}} = 2,000$ to cover the 1,000-iteration warmup and 1,000-iteration post-recovery horizon used in the stability experiment. Under this window, the seven cell means with $\Phi'(t) \leq 10^{-1}$ remained within the baseline $\pm 1\sigma_{\text{base}}$ band, whereas the five cells above the threshold deviated by 1.21–1.98 σ_{base} under always-hybrid recovery. The

TABLE III
DEFAULT TRAINING CONFIGURATION.

Parameter	Value	Parameter	Value
Layers	48	Experts	128
Hidden size	2048	MoE top- k	8
Attention heads	32	MoE FFN hidden	768
Query groups	4	Load balancing	aux. loss
Sequence length	4096	Aux. coef.	1×10^{-3}
Global batch	64	Optimizer	AdamW
Precision	BF16	LR/min LR	$10^{-4}/10^{-5}$
LR schedule	cosine	Weight decay	0.1
Gradient clipping	1.0		

TABLE IV
DEFAULT POLICY PARAMETERS.

Param.	Meaning	Default
Δ_{min}	Min. gap for hybrid recovery	1 iter
Δ_{max}	Max. single-event gap for hybrid recovery	200 iters
Φ_{max}	Max. staleness density in W_{exp}	1×10^{-1}
W_{exp}	exposure window	2 000 iters
N_{expert}	routed experts in exposure domain	128

TABLE V
GUARD-DISABLED BURST SWEEP: VALIDATION-LOSS DEVIATION UNDER ALWAYS-HYBRID RECOVERY (UNITS OF σ_{BASE}). SHADED CELLS EXCEED Φ_{max} AND TRIGGER RESTART UNDER THE DEFAULT POLICY.

$ F $	$\Delta=50$	$\Delta=100$	$\Delta=150$	$\Delta=200$
8	+0.32	+0.64	-0.05	+0.01
16	-0.18	+0.71	+1.42	+1.46
24	+0.59	+1.21	+1.98	+1.95

default policy therefore routes those five over-threshold cells to checkpoint restart.

D. RQ1: Single-Failure Recovery Cost and Mechanism Decomposition

RQ1 measures the low-latency recovery path: how much latency MoEGambit saves and whether its two mechanisms compose independently.

We injected hard failures after warmup and ran a 2×2 factorial crossing (i) **hybrid restore** versus **full checkpoint restart** with (ii) **two-phase** versus **single-phase** optimizer attachment. Each cell aggregated 500 injected recovery events; Table VI reports the mean. We measured latency from failure detection until the replacement rank produced its first post-recovery iteration.

Decomposition. The *hybrid restore* main effect was -5.50 s (-15.1%) and the *two-phase* main effect was -2.00 s (-5.5%), with an interaction of 0.35 s (0.96% of baseline, within noise). The two mechanisms were nearly additive in this experiment, jointly yielding the 28.914 s best cell—a 20.6% **reduction** over full checkpoint restart.

Result. Hybrid restore replaces NVMe-bound full-tensor restoration with peer-pulled non-expert state plus one expert-shard read, reducing I/O from $O(\text{global checkpoint})$ to $O(|E_{\text{new}}| \cdot \text{shard})$. Two-phase recovery overlaps optimizer-state restoration with resumed forward/backward; in our measurements, this overlap saved a nearly constant ≈ 2 s. The small interaction (0.96%) indicated that the mechanisms target different recovery segments.

Because Restart and MoC-System resume from checkpoint state, both replay the expected gap $\Delta=100$ iterations to reach step t ($T_{\text{replay}} \approx 1031$ s). Using the best-case load cost above, the replay-inclusive cost was 1067.4 s; MoEGambit resumed at step t directly, giving $1067.4/28.9 = 36.9\times$ replay-inclusive

TABLE VI
SINGLE-FAILURE TIME-TO-RESUME (2×2 FACTORIAL; MEAN PER EVENT, SECONDS, EXCLUDES REPLAY).

Restore path	Single	Two-phase	Δ (s)
Full checkpoint	36.417	34.238	-2.179
Hybrid (selective)	30.743	28.914	-1.829
Δ hybrid (s)	-5.674	-5.324	

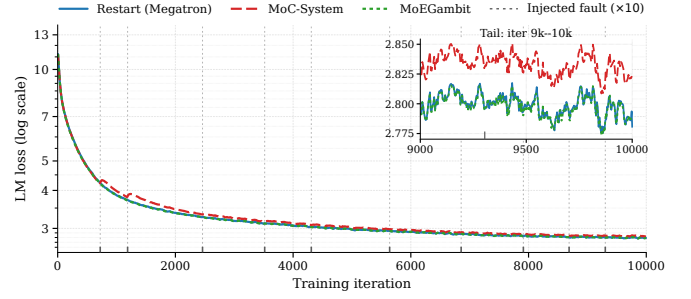


Fig. 3. Training-loss trajectories over 10,000 iterations with ten injected faults (gray verticals). Last-200-iteration mean: Restart 2.7919, MoEGambit 2.7910, MoC-System 2.8254.

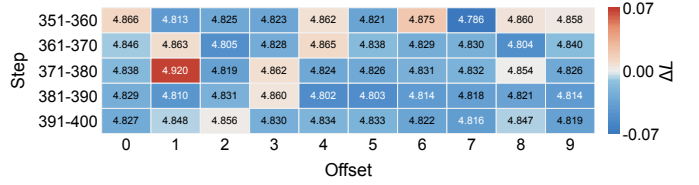


Fig. 4. Eval-loss heatmap for 50 single-failure recoveries that satisfied the contract. Each cell is one run; color encodes deviation from the 10-run NoFault mean ($\mu_{\text{base}} = 4.8543$, $\sigma_{\text{base}} = 0.024$). The distribution is centered near the baseline, with $44/50$ cells within $\pm 2\sigma_{\text{base}}$ and $23/50$ within $\pm 1\sigma_{\text{base}}$.

speedup for this 100-iteration gap. We report raw and replay-inclusive metrics separately; only the latter scales with the replay gap.

E. RQ2: Training Quality and Stability

RQ2 tests the quality side of the recovery contract: does hybrid recovery permitted by the contract avoid detectable degradation relative to failure-free training?

Setup. Each run performed 1,000 warmup iterations, injected one failure, and then continued for 1,000 iterations against paired NoFault and Restart runs. Gaps that satisfied the contract ($\Delta \in \{64, 128\}$) used the hybrid path; larger forced gaps $\Delta \in \{256, 512, 1000, 1500\}$ stressed the per-event guard and routed to restart unless the guard was deliberately disabled.

Result. For tested contract-satisfying hybrid gaps, paired validation loss and perplexity showed **no statistically detectable degradation** relative to NoFault/Restart. Fig. 3 shows the full 10,000-iteration trajectory under ten injected faults, and Fig. 4 zooms into 50 single-failure runs around the NoFault mean that satisfied the contract: **44/50** runs fell within $\pm 2\sigma_{\text{base}}$ and $23/50$ within $\pm 1\sigma_{\text{base}}$. A Wilcoxon signed-rank test on the 10 paired eval-loss differences at iter 600 yielded $p = 0.63$ and Cliff's delta = 0.07, giving no evidence of degradation. We used larger forced gaps only as guard stress tests; under the default policy, those cases route to restart.

Because runs are paired by seed, data order, checkpoint, and failure time, the criterion is conservative: MoEGambit is judged by paired deviation from the baseline trajectory, not by whether a single noisy run improves a score.

The heatmap targets isolated rank/GPU faults, a common production case emphasized by LLM traces and resilient-training systems [1], [2], [19]. As larger jobs increase EP or shard experts more finely, one failed GPU owns a smaller expert fraction, so $\Phi'(t)$ charges less per-event exposure; RQ3 covers bursts whose accumulated exposure triggers fallback.

Across all 10,000 iterations and injected faults, MoEGambit tracked Restart within run-to-run noise; MoC-System accumulated a small persistent gap consistent with PEC carrying older expert state across recovery boundaries.

Downstream zero-shot accuracy. Table VII reports Im-evaluation-harness [45] results at iter 10,000 on eight tasks (ARC-Easy [46], BoolQ [47], MathQA [48], Open-BookQA [49], PIQA [50], RACE [51], SWAG [52], Wino-Grande [53]).

Restart and MoEGambit agreed within ± 1.6 pp on every task. MoEGambit was numerically higher on average (+0.26 pp), but the gap was inside the per-checkpoint downstream noise floor [54]; we interpret it as **quality parity**. The small positive mean could reflect evaluation noise or mild dropout-like noise regularization from a bounded perturbation to a small expert subset [55], [56]. This interpretation is consistent with ST-MoE fine-tuning ablations where updating only non-MoE parameters performs comparably to updating all parameters, while sparse models tolerate some dropped-token noise [42]. It is **not evidence of improvement**. MoC-System reached 44.67 % (-0.39 pp vs. Restart) and trailed on six of eight probes, consistent with the persistent loss gap in Fig. 3.

F. RQ3: Multiple Failures and Expert-Weighted Staleness Density

RQ3 stress-tests the R2 specification: does the $\Phi'(t) \leq \Phi_{\max}$ guard separate low-exposure recoveries from higher-risk multi-rank bursts?

We stress-tested MoEGambit against the calibrated $\Phi_{\max} = 10^{-1}$ threshold using repeated and multi-rank burst failures. With `find_multi_fault.sh`, we swept $|F| \in \{8, 16, 24\}$ and $\Delta \in \{50, 100, 150, 200\}$ on the same 3×4 grid as Table V, plus repeated-window traces that revisit one rank within W_{exp} . This is a calibrated stress test, not an independent proof of generalization.

Result. Under the default policy, MoEGambit used hybrid recovery for the seven $\Phi'(t) \leq 10^{-1}$ cells and redirected the five over-threshold cells to checkpoint restart, where cell-mean loss returned inside the $\pm 1\sigma_{\text{base}}$ band. With the guard disabled, those same five cells reproduced the $1.21\text{--}1.98\sigma_{\text{base}}$ deviations in Table V. Thus, the absence of above-threshold degradation came from fallback, and the expert-weighted window captured risk missed by gap-only policies.

This also explains why $\Delta_{\max}$ alone is insufficient: equal-gap events can expose different expert fractions depending on failed ranks and recent repairs. $\Phi'(t)$ charges both gap and affected expert set, then ages the charge out of the window, separating isolated single-rank failures from dense bursts.

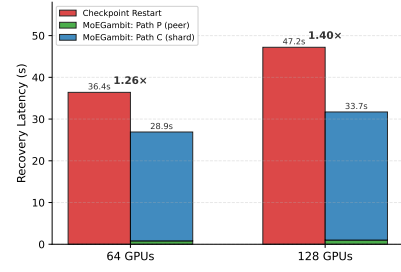


Fig. 5. Recovery latency at 64 and 128 GPUs. The Restart/MoEGambit ratio widened from $1.26\times$ to $1.40\times$ because hybrid I/O scales with the per-rank shard.

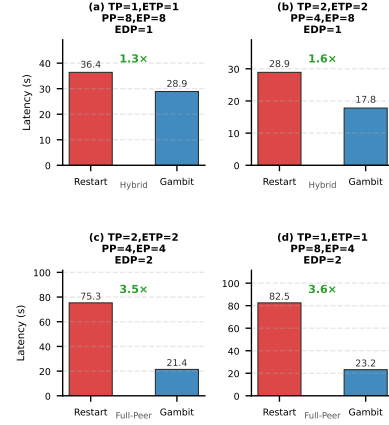


Fig. 6. Parallelism sensitivity at 64 GPUs using the Megatron Core EDP definition. EDP=1 layouts used hybrid recovery ($1.26\text{--}1.62\times$); EDP=2 enabled full-peer recovery and yielded $3.52\text{--}3.56\times$ speedups.

G. RQ4: Failure-Free Overhead

Result. Over 1,000 failure-free iterations, MoEGambit's hooks added $< 6\mu\text{s}$ per iteration and changed mean step time by only $+0.1\%$; a paired t -test did not reject a zero mean difference at $\alpha = 0.05$.

H. RQ5: Scalability

We reran the single-failure injection of §IV-D at 64 and 128 GPUs (dense $\text{TP}=1$, $\text{PP}=8$, $\text{EP} \in \{8, 16\}$, $\text{ETP}=1$; 10 seeds each) and tested four 64-GPU layouts spanning dense $\text{TP} \in \{1, 2\}$, $\text{ETP} \in \{1, 2\}$, $\text{PP} \in \{4, 8\}$, $\text{EP} \in \{4, 8\}$, and $\text{EDP} \in \{1, 2\}$. **Result.** Restart grew with global checkpoint size ($36.4\text{ s} \rightarrow 47.2\text{ s}$), while MoEGambit grew more slowly ($28.9\text{ s} \rightarrow 33.7\text{ s}$) because hybrid I/O scales with the per-rank shard. The ratio widened from $1.26\times$ to $1.40\times$ (Fig. 5), suggesting larger benefits at larger GPU scales.

Parallelism sensitivity. Across four 64-GPU layouts (Fig. 6), MoEGambit outperformed restart by $1.26\text{--}3.56\times$. The advantage was largest when $\text{EDP} \geq 2$ because a live expert peer lets MoEGambit restore both non-expert and expert state from peers; even at $\text{EDP}=1$, hybrid recovery remained faster by reading only the failed rank's expert shard from disk.

TABLE VII
ZERO-SHOT DOWNSTREAM ACCURACY (%) AT ITER 10,000 (LM-EVALUATION-HARNESS; STANDARD ERRORS ≤ 0.021).

System	ARC-E acc	BoolQ acc	MathQA n	OBQA n	PIQA n	RACE acc	SWAG n	WG acc	Avg.
Restart (Megatron)	46.72	59.88	22.08	29.00	68.34	29.00	55.17	50.28	45.06
MoC-System [23]	45.41	59.94	22.28	28.20	67.68	29.19	54.00	50.67	44.67
MoEGambit	48.36	58.56	21.54	30.60	68.17	29.28	55.43	50.59	45.32

* Per-column best is in **bold**. Metric row: “n” denotes acc_norm; others are acc.

I. RQ6: Cross-Model Generalization

To assess generalization beyond Qwen3-30B-A3B, we reran the core experiments on **DeepSeek-V2-Lite** [5] (15.7B total / 2.4B activated, 64 routed experts, 2 shared experts, top-6). Shared experts are dense-DP-replicated and used path P; routed experts used path C. Policy parameters were inherited from Table IV with $N_{\text{expert}}=64$.

Result. Restart took 26.09 s while MoEGambit recovered in 11.73 s, a 55.0% **reduction** (2.22 $\times$). Smaller expert shards made path C faster; the same burst-sweep grid did not require retuning $\Phi_{\text{max}} = 10^{-1}$ for this configuration, and routing shared experts through path C reduced the advantage by 3.2 pp.

J. Threats to Validity

Following standard SE validity categories [28], we summarize the main threats. *Internal validity.* Because no public MoC-System implementation exists, we used its best reported PEC configuration and timing numbers, reported raw latency separately from replay-inclusive speedup, and disabled PEC when measuring MoEGambit’s mechanisms. Paired seeds, identical data/failure steps, and NoFault σ_{base} bands controlled residual noise from collectives, networking, heterogeneity, storage, and scheduling. *Construct validity.* We measured 10,000-step loss/perplexity and eight zero-shot tasks, not full pre-training convergence; other uses may need longer runs or task-specific probes. *External validity.* Results cover two Megatron-LM MoE configurations on H20-3e interconnects; other layouts, fabrics, checkpoint backends, detectors, or auxiliary-loss settings may require recalibrating Φ_{max} .

V. RELATED WORK

Self-adaptive and self-healing systems. MAPE-K [25], Rainbow [26], and SE roadmaps [57], [58] argue that runtime repair should be specified, guarded, and auditable; MoEGambit applies this view to MoE recovery through $\Phi'(t)$, state-class boundaries, and logged reintegration.

Checkpointing, topology adaptation, and peer recovery. Checkpoint systems reduce the cost C in Young/Daly-style restart [39], [40], including CheckFreq [12], DeepFreeze [13], Check-N-Run [14], Gemini [15], REFT [16], and ByteCheckpoint [17], while preserving checkpoint-consistent recovery. Topology-adaptation systems [18]–[20], [59]–[61] reconfigure around surviving ranks but assume rank interchangeability.

FlashRecovery [11] pulls a dense-DP replica from a peer; MoEGambit extends peer recovery to dense-DP-replicated non-expert state and guards stale EP-sharded experts when EDP=1.

MoE systems and fault tolerance. MoE systems and models optimize routing, dispatch, and all-to-all efficiency [4], [6]–[10], [30]–[36], [62]; ST-MoE [42], sparse-upcycling [43], and V-MoE [63] develop MoE architectures. MoC-System [23] and MoEvent [24] are the closest MoE fault-tolerance systems, but remain checkpoint/log-based; MoEGambit targets recovery-side peer repair and runtime quality gating.

VI. DISCUSSION

Implications and deployment. Reliable MoE recovery should be state-aware and auditable. R1–R3 expose the policy predicate, debt $S(t)$, fallback reason, and quality trace, enabling audit-only deployment before hybrid repair is enabled inside the measured noise floor.

Complementarity with checkpointing. MoEGambit does not replace save-side checkpoint optimization. Lower checkpoint cost shortens restart and reduces expert-shard age, but checkpointing systems still return to step c . MoEGambit targets the work between c and failure step t : when admitted, peer state preserves current non-experts and the job resumes at t without replay.

Limitations and future work. MoEGambit requires a healthy dense-DP peer at the failed rank’s PP/dense-TP coordinate; failures that remove all eligible dense-DP peers fall back to restart. The evaluated $\Phi_{\text{max}} = 10^{-1}$ threshold assumes the same auxiliary-loss scale, static expert placement, and fail-stop failures; dynamic migration [35], slow degradation, silent data corruption [64], and network partitions require additional guards. Future work will add expert-usage statistics and online noise-floor estimates.

VII. CONCLUSION

MoEGambit makes MoE recovery state-aware and auditable, cutting raw latency by 20.6%–55.0% and achieving 36.9 $\times$ replay-inclusive speedup with **no statistically detectable degradation** in our tests. Its audit trail makes each fallback decision inspectable.

REFERENCES

- [1] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al., “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [2] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, Y. Jia, S. He, H. Chen, Z. Bai, Q. Hou, S. Yan, D. Zhou, Y. Sheng, Z. Jiang, H. Xu, H. Wei, Z. Zhang, P. Nie, L. Zou, S. Zhao, L. Xiang, Z. Liu, Z. Li, X. Jia, J. Ye, X. Jin, and X. Liu, “MegaScale: Scaling large language model training to more than 10,000 GPUs,” in *Proc. USENIX NSDI*, 2024, pp. 745–760.
- [3] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, “OPT: Open pre-trained transformer language models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [4] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. Le Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [5] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv preprint arXiv:2405.04434*, 2024.
- [6] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [7] Qwen Team, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [8] Ling Team, B. Zeng, C. Huang, C. Zhang, C. Tian, C. Chen, D. Jin, F. Yu, F. Zhu, F. Yuan, F. Wang, et al., “Every FLOP counts: Scaling a 300B mixture-of-experts LING LLM without premium GPUs,” *arXiv preprint arXiv:2503.05139*, 2025.
- [9] Ling Team, B. Han, C. Tang, C. Liang, D. Zhang, F. Yuan, F. Zhu, J. Gao, J. Hu, L. Li, M. Li, et al., “Every attention matters: An efficient hybrid architecture for long-context reasoning,” *arXiv preprint arXiv:2510.19338*, 2025.
- [10] Kimi Team, Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, et al., “Kimi K2: Open agentic intelligence,” *arXiv preprint arXiv:2507.20534*, 2025.
- [11] H. Zhang, J. Wang, Z. Yu, Y. Zhang, X. Ji, K. Mao, J. Zhang, Y. Zhang, T. Wu, F. Jie, X. Huang, Z. Cai, J. Cheng, S. Wang, W. Li, X. Bao, H. Xu, S. Zhao, J. Li, H. Sun, Z. Zhang, Y. Xiong, and C. Li, “FlashRecovery: Fast and low-cost recovery from failures for large-scale training of LLMs,” *arXiv preprint arXiv:2509.03047*, 2025.
- [12] J. Mohan, A. Phanishayee, and V. Chidambaram, “CheckFreq: Frequent, fine-grained DNN checkpointing,” in *Proc. USENIX FAST*, 2021, pp. 203–216.
- [13] B. Nicolae, J. Li, J. Wozniak, G. Bosilca, M. Dorier, and F. Cappello, “DeepFreeze: Towards scalable asynchronous checkpointing of deep learning models,” in *Proc. IEEE/ACM CCGrid*, 2020, pp. 172–181.
- [14] A. Eisenman, K. K. Matam, S. Ingram, D. Mudigere, R. Krishnamoorthi, K. Nair, M. Smelyanskiy, and M. Annavaram, “Check-N-Run: A checkpointing system for training deep learning recommendation models,” in *Proc. USENIX NSDI*, 2022, pp. 929–943.
- [15] Z. Wang, Z. Jia, S. Zheng, Z. Zhang, X. Fu, T. S. E. Ng, and Y. Wang, “GEMINI: Fast failure recovery in distributed training with in-memory checkpoints,” in *Proc. ACM SOSP*, 2023, pp. 364–381.
- [16] Y. Wang, X. Kang, S. Shi, X. He, Z. Tang, X. Pan, Y. Zheng, X. Wu, A. C. Zhou, B. He, and X. Chu, “Fault-tolerant hybrid-parallel training at scale with reliable and efficient in-memory checkpointing,” *arXiv preprint arXiv:2310.12670*, 2024.
- [17] B. Wan, M. Han, Y. Sheng, Y. Peng, H. Lin, M. Zhang, Z. Lai, M. Yu, J. Zhang, Z. Song, X. Liu, and C. Wu, “ByteCheckpoint: A unified checkpointing system for large foundation model development,” in *Proc. USENIX NSDI*, 2025, pp. 559–578.
- [18] J. Thorpe, P. Zhao, J. Eyolfson, Y. Qiao, Z. Jia, M. Zhang, R. Netravali, and G. H. Xu, “Bamboo: Making preemptible instances resilient for affordable training of large DNNs,” in *Proc. USENIX NSDI*, 2023, pp. 497–513.
- [19] I. Jang, Z. Yang, Z. Zhang, X. Jin, and M. Chowdhury, “Oobleck: Resilient distributed training of large models using pipeline templates,” in *Proc. ACM SOSP*, 2023, pp. 382–395.
- [20] S. Gandhi, M. Zhao, A. Skiadopoulos, and C. Kozyrakis, “ReCycle: Resilient training of large DNNs using pipeline adaptation,” in *Proc. ACM SOSP*, 2024, pp. 211–228. doi:10.1145/3694715.3695960.
- [21] C. Jin, Z. Jiang, Z. Bai, Z. Zhong, J. Liu, X. Li, N. Zheng, X. Wang, C. Xie, Q. Huang, W. Heng, Y. Ma, W. Bao, S. Zheng, Y. Peng, H. Lin, X. Liu, X. Jin, and X. Liu, “MegaScale-MoE: Large-scale communication-efficient training of mixture-of-experts models in production,” *arXiv preprint arXiv:2505.11432*, 2025.
- [22] Z. Yan, H. Bai, X. Yao, D. Liu, T. Liu, H. Liu, P. Li, E. Wu, S. Fan, L. Tao, R. Zhang, Y. Wang, S. Xu, J. Chang, X. Chen, K. Li, Y. Bai, G. Deng, N. Zheng, V. A. Korthikanti, et al., “Scalable training of mixture-of-experts models with Megatron Core,” *arXiv preprint arXiv:2603.07685*, 2026.
- [23] W. Cai, L. Qin, and J. Huang, “MoC-System: Efficient fault tolerance for sparse mixture-of-experts model training,” in *Proc. ASPLOS*, 2025, pp. 655–671. doi:10.1145/3676641.3716006.
- [24] S. Gandhi and C. Kozyrakis, “Sparse checkpointing for fast and reliable MoE training,” in *Proc. USENIX NSDI*, 2026, pp. 1395–1415.
- [25] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *IEEE Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [26] D. Garlan, S.-W. Cheng, A.-C. Huang, B. Schmerl, and P. Steenkiste, “Rainbow: Architecture-based self-adaptation with reusable infrastructure,” *IEEE Computer*, vol. 37, no. 10, pp. 46–54, 2004.
- [27] M. Salehie and L. Tahvildari, “Self-adaptive software: Landscape and research challenges,” *ACM Trans. Auton. Adapt. Syst.*, vol. 4, no. 2, pp. 14:1–14:42, 2009.
- [28] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.
- [29] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.
- [30] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “GShard: Scaling giant models with conditional computation and automatic sharding,” in *Proc. ICLR*, 2021.
- [31] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [32] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, “DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale,” in *Proc. ICML*, 2022.
- [33] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, J. Chau, P. Cheng, F. Yang, M. Yang, and Y. Xiong, “Tutel: Adaptive mixture-of-experts at scale,” in *Proc. MLSys*, 2023.
- [34] T. Gale, D. Narayanan, C. Young, and M. Zaharia, “MegaBlocks: Efficient sparse training with mixture-of-experts,” in *Proc. MLSys*, 2023.
- [35] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li, “FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models,” in *Proc. PPOPP*, 2022, pp. 120–134.
- [36] M. Zhai, J. He, Z. Ma, Z. Zong, R. Zhang, and J. Zhai, “SmartMoE: Efficiently training sparsely-activated models through combining offline and online parallelization,” in *Proc. USENIX ATC*, 2023, pp. 961–975.
- [37] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proc. SC*, 2021.
- [38] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A survey of rollback-recovery protocols in message-passing systems,” *ACM Comput. Surv.*, vol. 34, no. 3, pp. 375–408, 2002. doi:10.1145/568522.568525.
- [39] J. W. Young, “A first order approximation to the optimum checkpoint interval,” *Commun. ACM*, vol. 17, no. 9, pp. 530–531, 1974.
- [40] J. T. Daly, “A higher order estimate of the optimum checkpoint interval for restart dumps,” *Future Generation Computer Systems*, vol. 22, no. 3, pp. 303–312, 2006.
- [41] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *Proc. ICLR*, 2017.
- [42] B. Zoph, I. Bello, S. Kumar, N. Du, Y. Huang, J. Dean, N. Shazeer, and W. Fedus, “ST-MoE: Designing stable and transferable sparse expert models,” *arXiv preprint arXiv:2202.08906*, 2022.
- [43] A. Komatsuzaki, J. Puigcerver, J. Lee-Thorp, C. Riquelme Ruiz, B. Mustafa, J. Ainslie, Y. Tay, M. Dehghani, and N. Houlsby, “Sparse upcycling: Training mixture-of-experts from dense checkpoints,” in *Proc. ICLR*, 2023.

- [44] G. Penedo, H. Kydliček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf, “The FineWeb datasets: Decanting the web for the finest text data at scale,” in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024. arXiv:2406.17557.
- [45] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “A framework for few-shot language model evaluation,” Zenodo, version v0.4.0, 2023. doi:10.5281/zenodo.10256836.
- [46] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? Try ARC, the A12 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [47] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, “BoolQ: Exploring the surprising difficulty of natural yes/no questions,” in *Proc. NAACL-HLT*, 2019, pp. 2924–2936.
- [48] A. Amini, S. Gabriel, P. Lin, R. Koncel-Kedziorski, Y. Choi, and H. Hajishirzi, “MathQA: Towards interpretable math word problem solving with operation-based formalisms,” in *Proc. NAACL-HLT*, 2019, pp. 2357–2367.
- [49] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, “Can a suit of armor conduct electricity? A new dataset for open book question answering,” in *Proc. EMNLP*, 2018, pp. 2381–2391.
- [50] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, and Y. Choi, “PIQA: Reasoning about physical commonsense in natural language,” in *Proc. AAAI*, vol. 34, 2020, pp. 7432–7439.
- [51] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy, “RACE: Large-scale Reading Comprehension dataset from Examinations,” in *Proc. EMNLP*, 2017, pp. 785–794.
- [52] R. Zellers, Y. Bisk, R. Schwartz, and Y. Choi, “SWAG: A large-scale adversarial dataset for grounded commonsense inference,” in *Proc. EMNLP*, 2018, pp. 93–104.
- [53] K. Sakaguchi, R. Le Bras, C. Bhagavatula, and Y. Choi, “WinoGrande: An adversarial Winograd Schema Challenge at scale,” in *Proc. AAAI*, vol. 34, 2020, pp. 8732–8740.
- [54] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. van der Wal, “Pythia: A suite for analyzing large language models across training and scaling,” in *Proc. ICML*, 2023, pp. 2397–2430.
- [55] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [56] C. M. Bishop, “Training with noise is equivalent to Tikhonov regularization,” *Neural Computation*, vol. 7, no. 1, pp. 108–116, 1995.
- [57] B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, J. Magee *et al.*, “Software engineering for self-adaptive systems: A research roadmap,” in *Software Engineering for Self-Adaptive Systems*, LNCS 5525. Springer, 2009, pp. 1–26.
- [58] D. Weyns, “Software engineering of self-adaptive systems,” in *Handbook of Software Engineering*, S. Cha, R. N. Taylor, and K. Kang, Eds. Springer, 2019, pp. 399–443.
- [59] S. Athlur, N. Saran, M. Sivathanu, R. Ramjee, and N. Kwatra, “Varuna: Scalable, low-cost training of massive deep learning models,” in *Proc. EuroSys*, 2022, pp. 472–487.
- [60] J. Duan, Z. Song, X. Miao, X. Xi, D. Lin, H. Xu, M. Zhang, and Z. Jia, “Parcae: Proactive, liveput-optimized DNN training on preemptible instances,” in *Proc. USENIX NSDI*, 2024, pp. 1121–1139.
- [61] A. Qiao, A. Aghayev, W. Yu, H. Chen, Q. Ho, G. A. Gibson, and E. P. Xing, “Litz: Elastic framework for high-performance distributed machine learning,” in *Proc. USENIX ATC*, 2018, pp. 631–644.
- [62] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang, “DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models,” *arXiv preprint arXiv:2401.06066*, 2024.
- [63] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keyzers, and N. Houlsby, “Scaling vision with sparse mixture of experts,” in *Proc. NeurIPS*, 2021.
- [64] P. H. Hochschild, R. Govindaraju, and D. E. Culler, “Cores that don’t count,” in *Proc. USENIX HotOS*, 2021.